DATE

Template :

A template allow us to :

Write one piece of code that works with different data types.

Instead of writing same code again and again for int, float double etc. We write it once and C++ generates the correct version automatically.

Template means to make generalize.

Syntax :

```
template < typename placeholder >
```

or

```
template < class placeholder >
```

template → Keyword

Class / typename → Keyword

placeholder → generally it is T or can be any name

Types of templates in C++ :

There are two main types :

1. Function template
2. Class template

Function template :-

If template <....> is followed by a function definition → function template.

```
template <typename T>
T add (T a, T b)
{
    return a+b;
}

void main()
{
    cout << add(2,3);
    cout << add(2.5,3.5);
}
```

O/P :-
5
6

How C++ understands this internally :-

add(2,3);
compiler converts it to :-
int add (int, int);

add(2.5, 3.5)
compiler converts it to :
double add (double, double);

Or we can do Explicit template call :

```
add<int>(2,3);
add<double>(2.5,3.5);
```

Template with different data types :

```
template <typename T1, typename T2>
```

program:

```
template< typename T1, typename T2>
void show (T1 a, T2 b)
{
    cout<<a<<" "<<b<<endl;
}

show (10, 2.5)
show ("Hello", 5)
```

Class template :

If template <....> is followed by a function definition → function template.

Program :

```
template <class T>
class Box
{
    T value;
public:
    Box(T v)
    {
        value = v;
    }

    void show()
    {
        cout << value << endl;
    }
};

int main()
{
    Box<int> b1(10);
    Box<float> b2(3.14);

    b1.show();
    b2.show();
}
```